

Lab 6

Topic: Queues, Binary Trees and BSTs

Q1: Implementing a double ended queue .

Q2: Implementation of binary trees and using binary trees for expression processing and evaluation.

Q3: Implementation and use of BST

Note: There is no need to submit pseudo code and documentation for today's lab.

Q1 (30%)

A deque (double-ended queue) is a data-structure that extends a queue by supporting insertion and removal at both ends of the queue. In this assignment, you will use a circular array/vector to implement a Deque class and its operations below:

- **push_front(x)** : Add element x to the front of the deque. If the deque is full, print "Deque is full"

and do not perform the insertion.

- **push_back(x)** : Add element x to the back of the deque. If the deque is full, print "Deque is full"

and do not perform the insertion.

- **pop_front()** : Remove and print the front element from the deque. If the deque is empty, print "Deque is empty"

- **pop_back()** : Remove and print the back element from the deque. If the deque is empty, print "Deque is empty"

- **isEmpty()** : Check if the deque is empty and print True if it is, otherwise False.

Note: The implementation is similar to the circular array based implementation of queues discussed in the class. Feel free to handle the empty vs full issue with any of the methods discussed in the class or any other idea that you deem fit.

Input Format:

- The first input is an integer N representing the maximum size of the deque.
- The second input is an integer M representing the number of operations.
- Next, M lines where each line represents an operation in the following format:
 - **push front x**: Add element x to the front of the deque.
 - **push back x**: Add element x to the back of the deque.
 - **pop front**: Remove and print the front element of the deque
 - **pop back**: Remove and print the back element of the deque
 - **isEmpty**: Print True if the deque is empty, otherwise False.

Warning messages as indicated above are printed when the deque is full or empty.

Output Format:

- For each operation that produces output (pop_front, pop_back, isEmpty), print the result.

Constraints:

- Deque size (**N**) will be a positive integer ($1 \leq N \leq 10^4$).
- Number of queries (**M**) will be a positive integer ($1 \leq M \leq 10^5$).
- The operations **push front**, **push back**, **pop front**, **pop back** and **isEmpty** should be performed in $O(1)$ time complexity.
- The deque should operate using a circular array structure.

Examples:

Input 1:

4

7

push back 10

push front 20

push back 30

push front 40

2pop front

pop back

isEmpty

Output1 :

40

30

False

Explanation:

push back 10: Adds 10 to the back of the deque. Deque: [10]

push front 20: Adds 20 to the front. Deque: [20, 10]

push back 30: Adds 30 to the back. Deque: [20, 10, 30]

push front 40: Adds 40 to the front. Deque: [40, 20, 10, 30]

pop front: Removes 40 from the front. Deque after removal: [20, 10, 30]. Output: 40

pop back: Removes 30 from the back. Deque after removal: [20, 10]. Output: 30

isEmpty: Checks if the deque is empty. It's not, so output: False

Input2:

3

5

push back 5

push back 6

push back 7

push back 8

pop front

Output2:

Deque is full

5

Explanation:

push back 5: Adds 5 to the back. Deque: [5]

push back 6: Adds 6 to the back. Deque: [5, 6]

push back 7: Adds 7 to the back. Deque: [5, 6, 7]

push back 8: Attempt to add 8 fails since the deque is full. Output: "Deque is full"

pop front: Removes 5 from the front. Deque after removal: [6, 7]. Output: 5

Q2 (40%)

Implement the class `BinaryTreeNode` with the ability to store an int or char value. Using the `BinaryTreeNode`

Implement the class `BinaryTree` with suitable constructors and the following methods:

`Inorder()`: Outputs the inorder traversal of the tree

`Postorder()`: Outputs postorder traversal of the tree

`expr_tree(string)`: converts the string to an expression tree

`Infix()`: outputs the infix (fully parenthesised) expression represented by the tree.

Given a postfix expression, your task is to construct an expression tree from it. Once the tree is built, output both the infix and prefix expressions by performing inorder and preorder traversals, respectively.

Input Format:

- A single postfix expression as a string where operands are single-letter variables or single-digit integers, and operators are +, -, *.

Output Format:

 For each test case, print:

- The infix expression obtained through inorder traversal of the expression tree. The infix expression should be parenthesized.
- The prefix expression obtained through preorder traversal of the expression tree.

Constraints:

- $3 \leq \text{length of expression} \leq 100$
- All the operators used in expression are binary operators(need 2 operands).

Examples:

Input1:

ab+ef*g*-

Output1:

((a+b)-((e*f)*g))

- +ab**efg

Explanation:

Postfix Expression: ab+ef*g*-

Construct Expression Tree: Perform inorder traversal to get the infix expression: ((a + b) - ((e * f) * g)).

Perform preorder traversal to get the prefix expression: - + a b * * e f g.

Input2:

ab+cd+*

Output2:

((a+b)*(c+d))

*+ab+cd

Explanation:

Postfix Expression: ab+cd+*

Construct Expression Tree: Perform inorder traversal to get the infix expression: ((a + b) * (c + d)).

Perform preorder traversal to get the prefix expression: * + a b + c d.

Q3 (30%)

You are tasked with implementing a Binary Search Tree (BST) as a C++ class derived from the class `BinaryTree` defined in Q1. You should implement the following operations as member functions:

`Insert(x)` : Insert a key `x` into the BST.

`Delete(x)` : Remove a key `x` from the BST. Use In-order successor to replace the deleted element.

`IsBST()` : Determine if the tree represented is a BST or not.

Input Format:

- Each operation will either be an insertion `Insert X` or a deletion `Delete X`, where `X` is an integer value.
- the End line will always be printed for printing the BST in POST ORDER
- there might be insertion of duplicate elements to or removal of non-existent elements from the BST; these operations should be ignored without any error messages (recall that keys within a BST are unique).

Output Format:

- Print one line representing POST ORDER traversal of the BST.

Constraints:

- number of operations ≤ 1000 .
- value of nodes ≤ 1000 .

Example:

Input1:

```
Insert 7
Insert 3
Insert 5
Delete 3
Insert 6
Insert 8
Delete 9
Insert 4
print
```

Output1:

```
46587
```

Bonus (5 marks, capped to 5% total)

Write an algorithm to perform level-order traversal of a binary search tree (Breadth-First Traversal). You may use the class for binary tree in the previous questions. Input will be a sequence of instructions as in Q3. Output should be the level order traversal of the tree.

Example:

Input1:

Insert 7
Insert 3
Insert 5
Insert 6
Insert 8
Insert 4

Output1:

738546